

Google Sheets to Dashboard Automation Lab

Reduce Manual Updates Without Losing Governance

Google Sheets to Dashboard Automation Lab

Automation is valuable only when it reduces maintenance **without weakening data governance**. Begin with a manual workflow that you understand; automate after the definitions and update process are stable.

GenAI data guardrail: Use fictional, public, aggregated, or properly sanitized information with public generative AI tools. Do not enter student records, staff records, personally identifiable information, credentials, sensitive cybersecurity details, legal records, confidential procurement material, vendor-confidential information, or sensitive district project information unless your district has explicitly approved the tool and data use.

Three Maturity Levels

Level	Workflow	Good For	Risk/ Complexity
1. Manual	Export/verify data -> update data.md -> refresh dashboard	First prototype, monthly reporting	Lowest
2. Sheet-assisted	Maintain approved KPI table in Google Sheets -> export/copy structured data	Teams that prefer spreadsheet ownership	Moderate
3. Automated	Approved script/API updates Sheet or dashboard feed on schedule	Stable public/internal sources with clear ownership	Highest

Recommended Sheet Structure

Use one row per KPI or detail record. Avoid decorative merged cells.

name	category	current	target	unit	direction	status	owner	updated	note
Network Availability	Technology Operations	99.94	99.90	%	higher	On Track	Infrastructure	2026-08-20	Stable
SLA Compliance	Support Desk	88	90	%	higher	Watch	Help Desk	2026-08-20	Improving

Manual Sheet-to-data.md Prompt

Convert the sanitized table below into a Markdown table for a K-12 KPI dashboard. Preserve every value exactly. Do not infer or recalculate status. List any missing, duplicate, or inconsistent fields under a separate QA heading.

[PASTE TABLE]

When a Public Data Source Can Be Automated

The TCEA data-display workflow notes that generative AI can help write Google Apps Script to retrieve a **public** web table into a Sheet and refresh it on a trigger. Treat that as a software-development task:

1. Confirm the source permits automated access.
2. Confirm no credentials or restricted data are embedded in code.
3. Define how duplicates, missing rows, timeouts, and source changes are handled.
4. Log update time and failures.
5. Test in a copy of the Sheet.
6. Have a human review the generated code.
7. Establish an owner for the trigger and script.

Planning Prompt Before Writing Code

I have a Google Sheet used for a K-12 dashboard and an approved PUBLIC source. Do not write code yet.

Source URL and format: [SOURCE]
Destination sheet/tab: [TAB]
Columns needed: [COLUMNS]
Refresh cadence: [CADENCE]
Unique row key: [KEY]

Propose an Apps Script design that includes:

- retrieval method;
- parsing approach;
- duplicate prevention;
- error handling;
- update log;
- rate-limit/terms questions to verify;
- time-driven trigger plan.

Flag any assumptions for human approval.

Code-Generation Prompt After Approval

Write Google Apps Script from this approved design:
[APPROVED DESIGN]

Requirements:

- configuration variables at the top;
- no hard-coded credentials;
- preserve header row;
- do not delete existing rows unless the approved design requires it;
- prevent duplicates using the approved key;
- log success/failure and timestamp;
- include comments and setup instructions;
- include steps for a time-driven trigger;
- do not publish or share the Sheet automatically.

Dashboard Feed Options

Option A - Keep data.md as the Controlled Publication Layer

The Sheet is the working source. A designated person exports/updates `data.md` after verification. This is simple, reviewable, and appropriate when the dashboard is public.

Option B - Fetch an Approved Public CSV/JSON Feed

A dashboard can fetch a published data feed, but publishing a Sheet can expose more information than intended. Use only when the data is approved for public access and the URL does not reveal restricted content.

Option C - Internal Web App/API

For internal data, use an approved authenticated method rather than making the source public for convenience. Coordinate with district security, privacy, legal, and platform administrators.

Automation QA Checklist

- ☐ Source terms permit automated retrieval.
- ☐ Data classification is documented.
- ☐ Script contains no secrets.
- ☐ Source schema changes fail visibly rather than silently.
- ☐ Duplicate prevention is tested.
- ☐ Failed updates generate a visible log/alert.
- ☐ Last successful refresh is displayed.
- ☐ A human owner can disable the automation.
- ☐ Dashboard shows stale-data warning when appropriate.
- ☐ Public-release review remains separate from technical automation.

Source and Alignment Notes

This sample library is designed as a companion to **Module 8: Project Management and Change Leadership** in the TCEA Technology/IT Director Certification. It extends the module's emphasis on desired outcomes, progress reporting, KPIs, dashboards, communication, change adoption, and responsible generative AI use.

Useful reference points:

- Make Data Displays Without Writing Code - TCEA TechNotes article on GenAI-assisted data displays, dashboard workflows, Google Sheets, GitHub Pages, and embedding.
- KPI Dashboards Built with Gen AI - TCEA TechNotes article demonstrating a browser-based dashboard paired with an editable Markdown data file.
- Metrics and KPIs Made Easy - TCEA TechNotes introduction to actionable KPI scorecards for technology leaders.
- K-12 KPI Dashboard Generator - browser-based KPI planning and dashboard starter.
- District KPI Dashboard Example - example of a scoreboard-style district dashboard.

- Data Display Patterns Field Guide - examples, prompts, and hosting considerations for lightweight visual displays.

Data note: Every Ardent ISD number in this library is illustrative sample data. Replace it with locally verified data before operational use.